

Kernel Bypass Networking for High-Frequency Trading

Why standard TCP/IP isn't fast enough — and what HFT uses instead



Presenter: Protocol Lead
CN Module 5 — Group 07
Analyzing the network latency of High Frequency Trading

1 μ s

Millions



In HFT, a one-microsecond advantage can translate into millions of dollars of profit or loss.



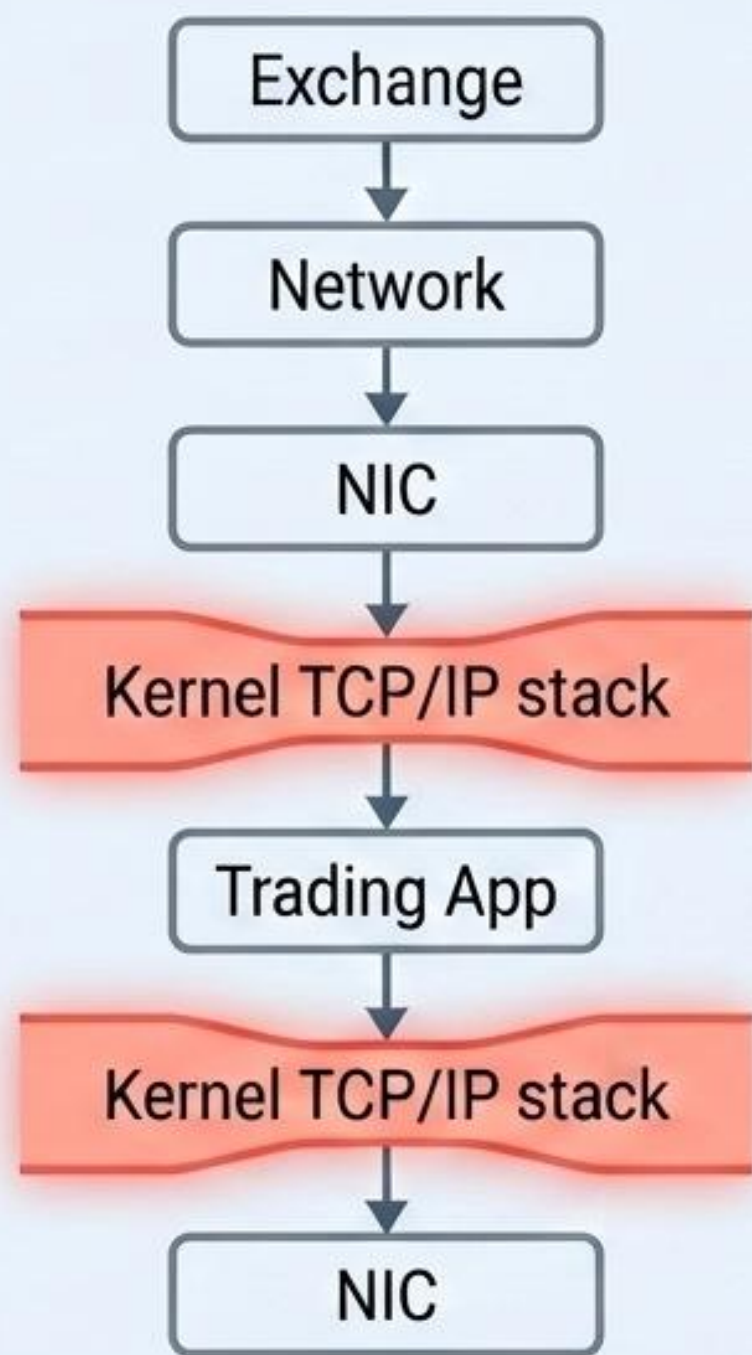
Trading is now dominated by algorithmic execution; orders are placed by machines competing at the physical limits of network speed.



Two physical limits dominate: propagation delay (distance) and queuing delay (congestion).



This presentation answers: where does latency come from, and how does the industry reduce it at the software layer?



Stage	Approximate latency	Type
Network propagation	$\approx 1 \text{ ms per } 200 \text{ km}$	Physical
NIC receive (DMA)	$\approx 1\text{--}3 \mu\text{s}$	Hardware
Kernel TCP/IP stack	$\approx 10\text{--}30 \mu\text{s}$	Software (bottleneck)
Trading application logic	$\approx 1\text{--}5 \mu\text{s}$	Software
Kernel + NIC transmit	$\approx 10\text{--}30 \mu\text{s}$	Software (bottleneck)

Latency values are approximate ranges typical of industry benchmarks; actual numbers depend on NIC, CPU, and system tuning.

Key Takeaway: The trading strategy itself takes only a few microseconds. The kernel — entered twice, on receive and on transmit — wastes 20–60 microseconds in between. That is the software bottleneck.

Interrupts

every incoming packet triggers an interrupt, forcing the CPU to switch context

Memory copies

packet data is copied from kernel space to user space before the application can read it

Context switching

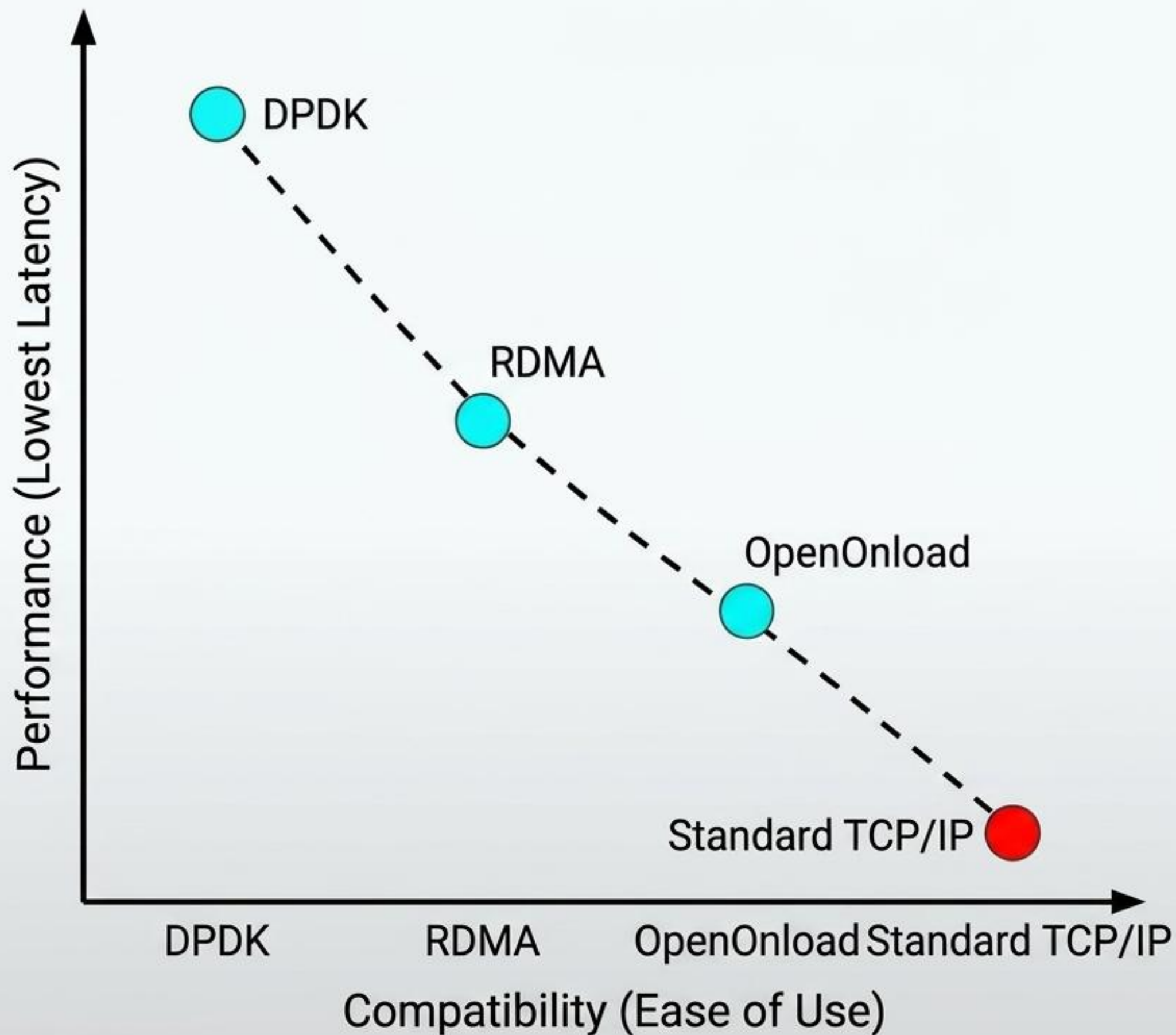
switching between kernel and user mode costs CPU cycles and cache pollution

Protocol processing

the TCP/IP stack itself runs significant logic: checksums, ACKs, congestion control, retransmits



Conclusion: If the kernel is the bottleneck, the answer is not to make the kernel faster — **it is to skip the kernel entirely.** This is called **kernel bypass.**

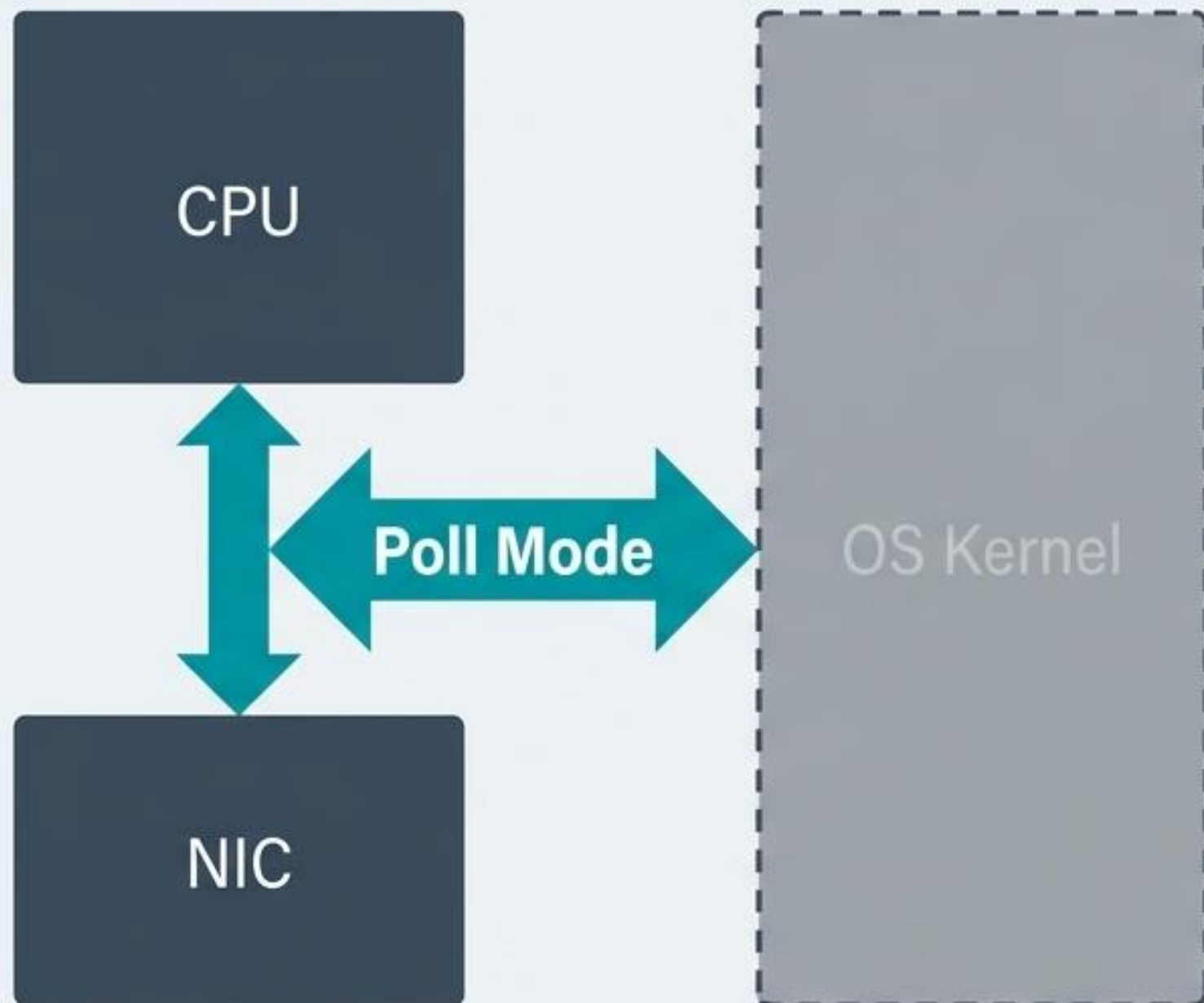


DPDK is the ultimate tool for hand-crafting your own packet processing pipelines, from the NIC to your application. It gives you complete control.

RDMA skips the kernel TCP/IP stack and the socket copy entirely, allowing direct memory access between applications across the network.

OpenOnload provides a user-space TCP/IP stack that is transparently compatible with existing socket applications.

All three skip the kernel TCP/IP stack and the socket copy. They differ in how far they bypass and what they require in return.



Data Plane Development Kit (DPDK) — an open-source library that moves packet processing from the kernel to user space. The NIC writes packets directly into user-space buffers; the application processes them with no kernel involvement.

Three key mechanisms

Poll Mode Driver (PMD) — the CPU continuously polls the NIC instead of waiting for interrupts. Eliminates interrupt latency entirely.

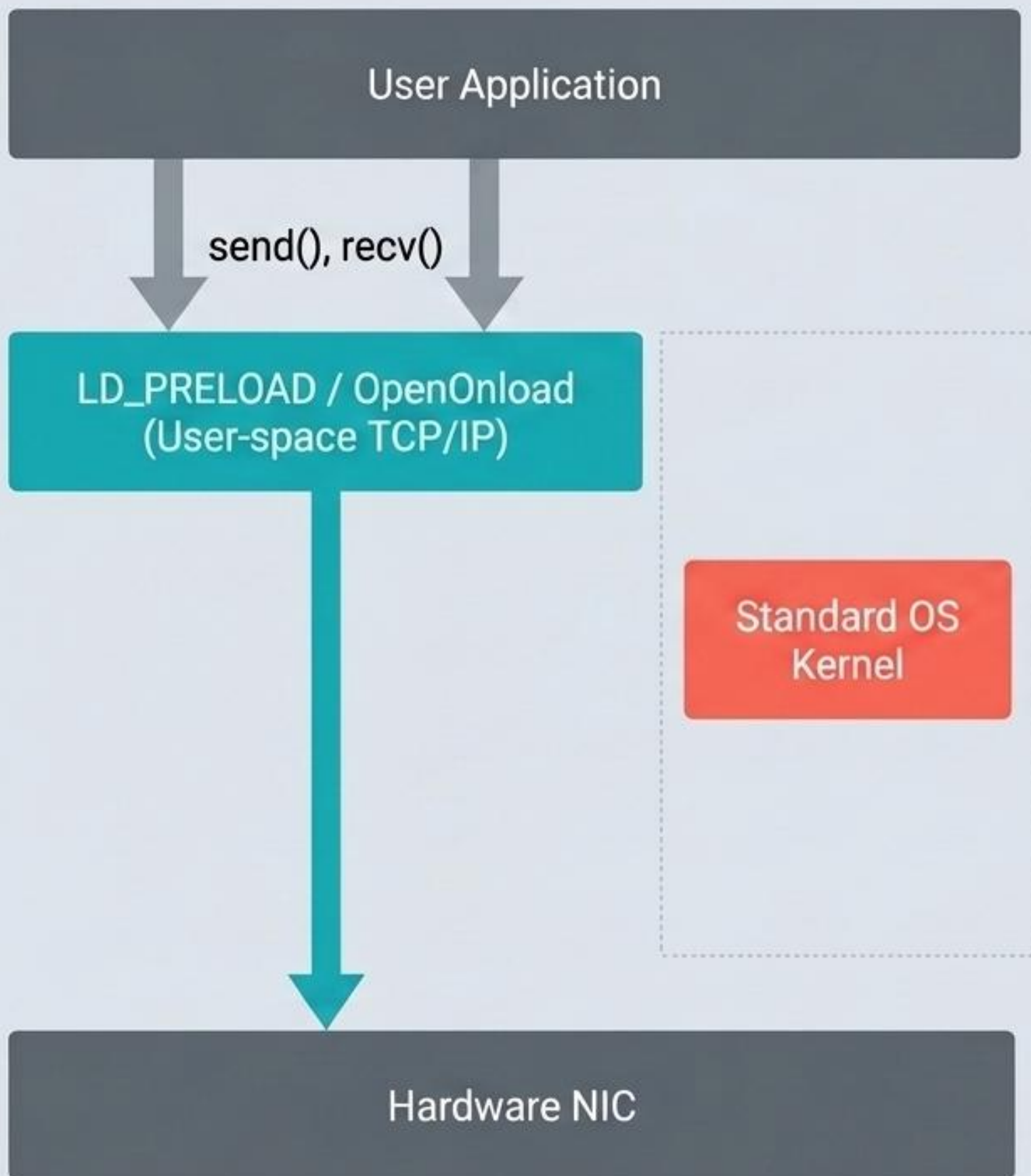
Huge Pages — uses 2 MB or 1 GB memory pages instead of the standard 4 KB. Reduces TLB misses and speeds up memory access.

CPU core pinning + NUMA awareness — dedicates specific cores to packet processing and keeps memory access local to the CPU socket. Maximizes cache efficiency.

Trade-offs

- A core is consumed at 100 percent even when idle — polling never sleeps. High power draw.
- Standard socket API is unavailable; applications must be rewritten against the DPDK API.
- Requires DPDK-supported NICs.

Latency: Sub-microsecond — the lowest software latency of the three.



OpenOnload

OpenOnload — a user-space TCP/IP stack developed by Solarflare, now part of AMD/Xilinx. It implements TCP and UDP entirely in the user process while exposing the standard Linux socket API.

Key characteristics

- A Drop-in compatibility** — applications use the same socket calls (send, recv, epoll). OpenOnload is loaded via LD_PRELOAD, intercepting socket calls transparently.
- B Lower-level APIs available** — for more performance, applications can use TCPDirect or ef_vi to access the NIC at a lower level.
- C Industry adoption** — widely deployed across the financial industry, including major exchanges and trading firms.

Trade-offs & Performance

- Tied to Solarflare/AMD NIC hardware. Slightly higher latency than DPDK because the full TCP state machine still runs.
- Latency: Approximately **1–3 microseconds**.

Frictionless Path



Remote Direct Memory Access (RDMA) — a technology that allows one machine to read or write memory on another machine directly, without involving the remote CPU or operating system. The NIC handles the transfer end-to-end.

Three core principles

- **Zero-copy** — data moves from local to remote memory in a single DMA transfer.
- **Kernel bypass** — applications submit work directly to the NIC using the verbs API.
- **CPU offload** — the remote CPU is not interrupted.

Three implementations

- InfiniBand (dedicated fabric)
- RoCE (RDMA over Ethernet)
- iWARP (RDMA over TCP)

Use in HFT

- Used inside the data center: market data distribution, order book replication, and inter-process communication.

Latency: Approximately 1–2 microseconds on InfiniBand or RoCE.

Low-Latency Networking: A Comparative Matrix

Dimension	Standard TCP/IP	OpenOnload	DPDK	RDMA
Latency	30–60 μ s	1–3 μ s	< 1 μ s	1–2 μ s
Code change	None	None (LD_PRELOAD)	Rewrite required	New API (verbs)
Hardware	Any NIC	Solarflare only	DPDK-supported NIC	RDMA NIC + fabric
Kernel bypass?	No	Yes	Yes	Yes (and CPU bypass)
Best for	General-purpose servers	Migrating existing trading systems	Greenfield ultra-low-latency systems	Intra-datacenter high-performance comm.

Conclusion: The more of the kernel you bypass, the faster you go — and the more your code and hardware must change.

Conclusion: The Frictionless Path

1 The dominant software bottleneck in HFT is the kernel TCP/IP stack — twenty to sixty microseconds wasted per round trip.

2 DPDK, OpenOnload, and RDMA all bypass this bottleneck, but with different philosophies: poll-mode packet processing, transparent socket replacement, and direct remote memory access.

3 The choice is a trade-off between compatibility, performance, and hardware dependency. There is no single best answer.

Connecting to the team project

This protocol-level analysis complements the team's coverage of physical-layer optimizations (co-location, microwave links) and economic context (the HFT arms race). Together, they explain why latency reduction is pursued from every direction at once.

References

Lockwood, J. et al. A Low-Latency Library in FPGA Hardware for High-Frequency Trading (HFT). IEEE Symposium on High-Performance Interconnects, 2012.

Postel, J. Transmission Control Protocol. IETF RFC 793, September 1981.

DPDK Project. DPDK Programmer's Guide. Available at doc.dpdk.org.

AMD / Xilinx (formerly Solarflare). OpenOnload User Guide and TCPDirect / ef_vi Programmer Reference.

NVIDIA (formerly Mellanox). RDMA Aware Networks Programming User Manual.

Aquilina, M., Budish, E., and O'Neill, P. Quantifying the High-Frequency Trading Arms Race. The Quarterly Journal of Economics, Vol. 137, No. 1, 2022.

Bhattacharjee, D. et al. A Bird's Eye View of the World's Fastest Networks. ACM Internet Measurement Conference (IMC), 2020.